Neural Posterior Estimation via Normalizing Flow

Hyungsuk Tak
Department of Statistics
Department of Astronomy and Astrophysics
Institute for Computational and Data Sciences
Pennsylvania State University

The core goal of data-driven science is to extract useful information from data, and the likelihood function is the most fundamental tool for allowing the data to inform unknown quantities of interest. In some cases, however, we encounter problems where evaluating the likelihood function is computationally too expensive or even impossible. The former occurs when the data set contains too many observations (rows) or features (columns), or when the model itself is overly complex. In such cases, likelihood-based inference procedures such as maximum likelihood estimation, likelihood-ratio tests, and Bayesian posterior inference can become computationally prohibitive. The latter case arises when the likelihood function involves intractable integrals and cannot be expressed in closed form. For example, in reverberation mapping time-lag estimation problems, the likelihood function for the unknown model parameters can be constructed from the observed flux data.

In this case, direct likelihood-based inference becomes infeasible. As a result, practitioners often adopt approximation techniques to bypass exact likelihood evaluation, such as Monte Carlo integration, Riemann sum, or approximate Bayesian computation (so-called ABC), one of the most widely used classical likelihood-free methods.

Simulation-based inference (SBI; Cranmer et al. 2020 for a review) is a machine-learning-based alternative for likelihood-free inference and is applicable to both frequentist and Bayesian frameworks via neural likelihood estimation (Cranmer et al., 2016) and neural posterior estimation (Papamakarios & Murray, 2016; Papamakarios et al., 2019), respectively. The appeal of SBI lies in its minimal requirement: it is feasible whenever data can be simulated from a model of interest. In fact, SBI can be applied not only to settings where likelihood evaluation is expensive or infeasible, but also more broadly to any model-based inference problem as long as simulated data sets can be repeatedly generated.

The data simulation process will be explained using a simple regression setting for estimating the Hubble constant with data from 24 extragalactic nebulae (Hubble, 1929), as described during the lecture on statistical inference. The model and the likelihood function are as follows.

For Bayesian inference, we need to specify prior distributions for the unknown model parameters. The most common choice in simulation-based inference is a jointly uniform prior distribution (i.e., a hypercube) to reflect our complete prior ignorance about these parameter values.

Then, our Bayesian model can be described hierarchically.

The goal of Bayesian inference is to obtain the posterior distribution of θ given data x, whose density function is denoted by $\pi(\theta \mid x)$. In this simple setting of the Hubble constant estimation, the target posterior density function is analytically tractable and available in closed form.

However, in more realistic settings where the likelihood function is intractable or computationally expensive, the central question in neural posterior estimation is whether we can learn the posterior density, $\pi(\theta \mid x)$, that is, a mapping from x to θ, indirectly from simulated pairs (x, θ) without evaluating the likelihood function $L(\theta)$. For illustrative purposes, let us assume that we cannot compute $L(\theta)$, but we can simulate pairs from the model.

The hierarchical model specification defines a generative model that describes how we simulate the data. We first generate parameter values from their prior distributions, $\pi(\theta)$:

And then, given each sampled parameter value, simulate data from the model, $f(x \mid \theta)$:

Each simulated parameter value and its corresponding data set, denoted by x, form a pair sampled from their joint distribution. With M simulated pairs,

The reason we can treat these pairs as samples drawn from the joint distribution is based on the principle of Monte Carlo simulation:

This expression (introduced in the lecture for probability) shows that we can sample (x, θ) from their joint distribution $\pi(x, \theta)$ by first sampling θ from its marginal distribution (the prior, $\pi(\theta)$), and then sampling x from the conditional distribution given θ, i.e., $f(x \mid \theta)$.

Neural posterior estimation via normalizing flows

Normalizing flows are a class of generative models that construct complex probability distributions by transforming a simple base distribution through a sequence of invertible and smooth mappings. For example,

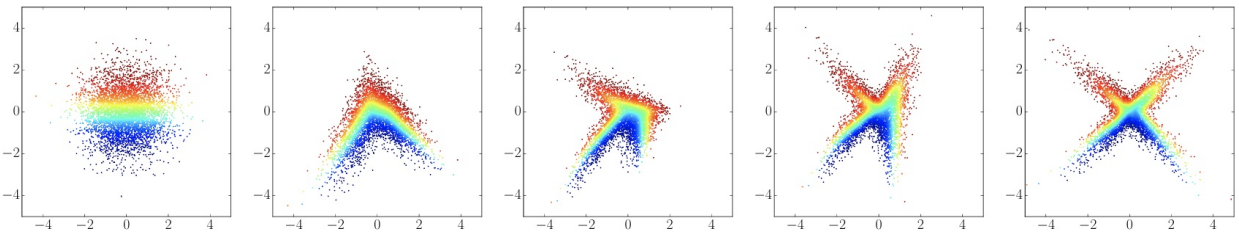


Image credit: Figure 1 of Papamakarios et al. (2019)

Typically, we begin with a latent variable z drawn from a tractable distribution such as the standard Gaussian distribution, $N(0, 1)$,

whose density is known in closed form:

We then apply a sequence of bijective transformations $g_1, g_2, \ldots, g_K$ to obtain $\theta = g_K \circ \cdots \circ g_1(z)$. Each transformation is parameterized. For example, if g_1 is a location-scale transformation,

Similarly, $g_2, \ldots, g_K$ have their own sets of parameters, denoted by $\phi_2, \ldots, \phi_K$. Let us collectively denote all transformation parameters by $\phi = \{\phi_1, \ldots, \phi_K\}$ and write the overall transformation as g_ϕ, i.e.,

In a normalizing flow, these transformation-related parameters, ϕ, are modeled as an output

vector of a neural network that takes data x as input, i.e., $\phi = h_w(x)$, where h_w denotes a complicated but deterministic function defined by an architecture of a neural network and w denotes the unknown network weights. Since each transformation is invertible, the corresponding density of the transformed parameters θ can be computed exactly using the change-of-variables formula:

This is called the flow-based density function of θ given x. The goal is to choose ϕ (equivalently, w, because $\phi = h_w(x)$) so that $q_\phi(\theta \mid x)$ is close to the true posterior $\pi(\theta \mid x)$.

Closeness is typically measured using the Kullback–Leibler (KL) divergence:

Although it is ideal to compute this KL divergence in principle, we cannot evaluate it exactly because the target posterior density is assumed to be intractable (due to the intractable likelihood function). Since this KL divergence cannot be evaluated directly, we introduce a trick that averages this KL divergence over simulated data, as an alternative:

Minimizing this average KL divergence with respect to ϕ (or ω) is equivalent to minimizing another KL divergence because both KL divergences are minimized when $q_\phi(\theta \mid x) = \pi(\theta \mid x)$ (i.e., at the same values of ϕ or ω):

The normalizing flow treats the latter KL divergence as the loss function to be minimized because it can be approximately evaluated using the simulated pairs, $(x^{(i)}, \theta^{(i)}), i = 1, \ldots, M$.

Consequently, we can optimize the neural network weights w by minimizing this approximate loss function:

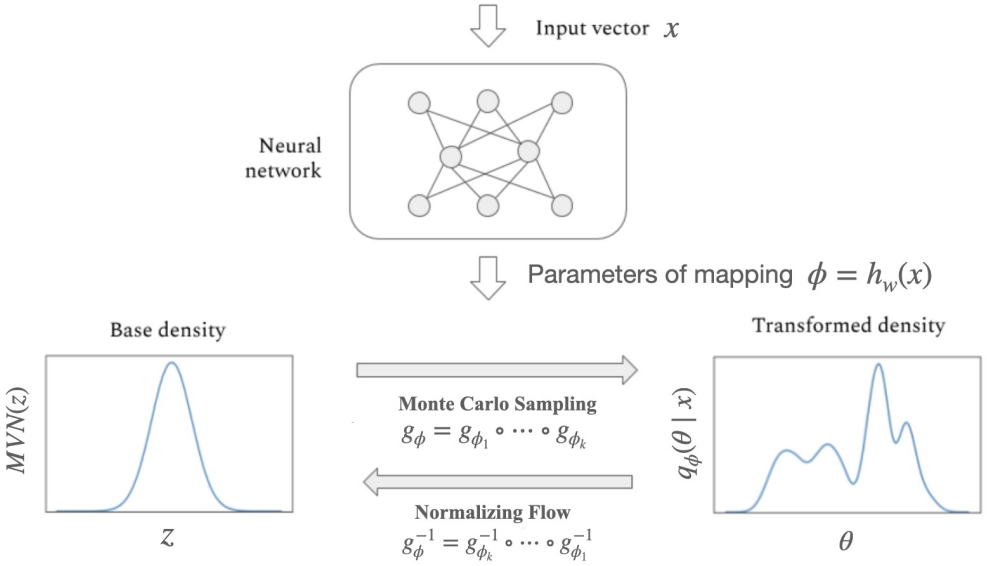
Once trained by M simulated pairs (i.e., after learning ω from the training data sets), the learned transformation $g_{\hat{\phi}}$ allows us to generate approximate posterior samples given the observed data x_{obs} by first sampling

and then applying the transformation,

For example, if we aim to draw 10,000 ($\neq M$) posterior samples of θ given x_{obs}, this procedure is repeated 10,000 times: we draw 10,000 random vectors of z (each of length m), compute $\theta = g_{\hat{\phi}}(z)$ for each z (where $\hat{\phi} = h_{\hat{w}}(x_{\text{obs}})$), and treat these as 10,000 posterior samples of θ drawn from $\pi(\theta \mid x_{\text{obs}})$.

As a byproduct, the resulting flow-based density, $q_{\hat{\phi}}(\theta \mid x_{\text{obs}})$, serves as an approximate posterior density function, $\pi(\theta \mid x_{\text{obs}})$.

The neural posterior estimation via normalizing flow is summarized in the following diagram:



A key design challenge in normalizing flows is to construct transformations that are expressive while maintaining computational tractability of the Jacobian determinant. Popular approaches include affine coupling layers (e.g., RealNVP; Real-valued Non-Volume Preserving), which induce triangular Jacobians, autoregressive flows, and continuous-time flows based on neural ordinary differential equations. These methods enable flexible density modeling and have been widely used in density estimation, variational inference, and generative modeling.